```
$foo = "10.0 pigs " + 1.0;          // $foo is float (11)
?>
```

To test any of the examples, cut and paste the examples and insert the following line to see what's going on:

```
<?php
echo "\$foo==$foo; type is " . gettype ($foo) . "<br />\n";
?>
```

Do not expect to get the code of one character by converting it to integer, as is done in C. Use the ord() and chr()functions to convert between ASCII codes and characters.

4.2.6 Details of the String Type

The string in PHP is implemented as an array of bytes and an integer indicating the length of the buffer. It has no information about how those bytes translate to characters, leaving that task to the programmer. There are no limitations on the values the string can be composed of; in particular, bytes with value *0* ("NUL bytes") are allowed anywhere in the string (however, a few functions, said in this manual not to be "binary safe", may hand off the strings to libraries that ignore data after a NUL byte.)

This nature of the string type explains why there is no separate "byte" type in PHP – strings take this role. Functions that return no textual data – for instance, arbitrary data read from a network socket – will still return strings.

Given that PHP does not dictate a specific encoding for strings, one might wonder how string literals are encoded. For instance, is the string *"á"* equivalent to *"\xE1"* (ISO-8859-1), *"\xC3\xA1"* (UTF-8, C form), *"\x61\xCC\x81"* (UTF-8, D form) or any other possible representation? The answer is that string will be encoded in whatever fashion it is encoded in the script file.

Thus, if the script is written in ISO-8859-1, the string will be encoded in ISO-8859-1 and so on. However, this does not apply if Zend Multibyte is enabled; in that case, the script may be written in an arbitrary encoding (which is explicity declared or is detected) and then converted to a certain internal encoding, which is then the encoding that will be used for the string literals. Note that there are some constraints on the encoding of the script (or on the internal encoding, should Zend Multibyte be enabled) – this almost always means that this encoding should be a compatible superset of ASCII, such as UTF-8 or ISO-8859-1. Note, however, that state-dependent encodings where the same byte values can be used in initial and non-initial shift states may be problematic.Of course, in order to be useful, functions that operate on text may have to make some assumptions about how the string is encoded. Unfortunately, there is much variation on this matter throughout PHP's functions: